

YZM322 - Linux Temelleri - Ödev 4 Raporu

Docker Compose ile Çok Servisli Image Repository Uygulaması

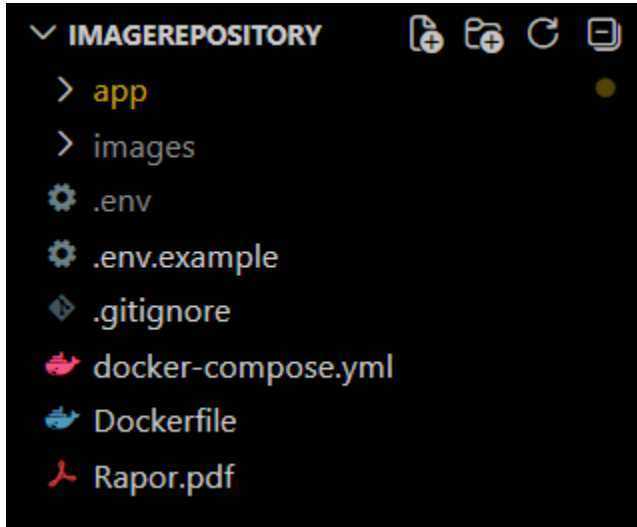
1. Giriş ve Proje Mimarisi

Bu rapor, Docker ve Docker Compose araçları kullanılarak izole ağ yapılandırmaları üzerinde entegre çalışan çok servisli bir Image Repository uygulamasının kurulum ve doğrulama aşamalarını içermektedir. Proje mimarisi; kullanıcı etkileşimini sağlayan bir web arayüzü (Flask), ilişkisel metadata veritabanı (PostgreSQL) ve nesne depolama servisinden (MinIO Object Storage) meydana gelmektedir.

Veri Depolama İlkesi: Resim dosyalarının fiziki içerikleri kesinlikle veritabanı container'ı içinde tutulmamış, MinIO üzerinde oluşturulan isimlendirilmiş bir bucket altında saklanmıştır. PostgreSQL veritabanında ise sadece resimlerin üstverileri (başlık, açıklama, orijinal dosya adı, dosya türü, dosya boyutu, benzersiz object key ve yüklenme tarihi) kalıcı volume yapıları aracılığıyla yönetilmektedir.

2. Yapılandırma ve Canlı Sistem Ekran Görüntüleri

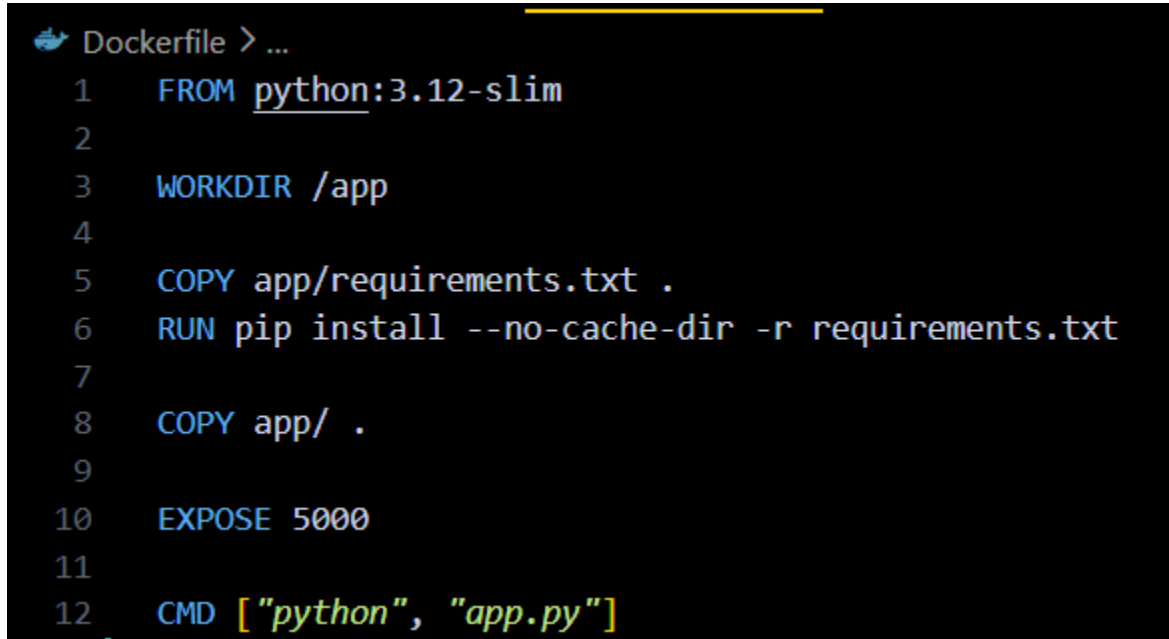
Görüntü 1: Proje Klasör Yapısı



Açıklama: Bu görüntü, projenin kök dizinindeki klasör ve dosya hiyerarşisini göstermektedir. app/ kaynak kod klasörü, images/ ekran görüntüleri klasörü, Docker

konfigürasyon dosyaları, .env.example ve Rapor.pdf uyumlu ve düzenli bir hiyerarşiyle listelenmiştir.

Görüntü 2: Dockerfile Dosyası İçeriği

A screenshot of a code editor showing the content of a Dockerfile. The editor has a dark background with a yellow title bar. The title bar contains a Docker logo and the text 'Dockerfile > ...'. The code is written in a light blue font and is numbered from 1 to 12 on the left side. The code defines a Docker image based on 'python:3.12-slim', sets the working directory to '/app', copies 'requirements.txt' and the application code, installs dependencies, and sets the command to run 'python app.py' on port 5000.

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY app/requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app/ .
9
10 EXPOSE 5000
11
12 CMD ["python", "app.py"]
```

Açıklama: Bu görüntü, web uygulaması imajının nasıl oluşturulduğunu gösteren Dockerfile içeriğidir. Temel imaj olarak python:3.12-slim seçilmiş, çalışma dizini ayarlanmış, bağımlılık kurulumları ve dosya kopyalama işlemleri tamamlanarak başlatma komutu girilmiştir.

Görüntü 3: docker-compose.yml Yapılandırması

```
docker-compose.yml
1 services:
2
3   web:
4     build: .
5     ports:
6       - "5000:5000"
7     environment:
8       - DATABASE_URL=postgresql://$${POSTGRES_USER}:$${POSTGRES_PASSWORD}@db:5432/$${POSTGRES_DB}
9       - MINIO_ENDPOINT=minio:9000
10      - MINIO_ACCESS_KEY=$${MINIO_ROOT_USER}
11      - MINIO_SECRET_KEY=$${MINIO_ROOT_PASSWORD}
12      - MINIO_BUCKET=$${MINIO_BUCKET}
13    depends_on:
14      - db
15      - minio
16    networks:
17      - appnet
18
19   db:
20     image: postgres:16
21     environment:
22       - POSTGRES_USER=$${POSTGRES_USER}
23       - POSTGRES_PASSWORD=$${POSTGRES_PASSWORD}
24       - POSTGRES_DB=$${POSTGRES_DB}
25     volumes:
26       - postgres_data:/var/lib/postgresql/data
27     networks:
28       - appnet
29
30   minio:
31     image: minio/minio:latest
32     command: server /data --console-address ":9001"
33     ports:
34       - "9000:9000"
35       - "9001:9001"
36     environment:
37       - MINIO_ROOT_USER=$${MINIO_ROOT_USER}
38       - MINIO_ROOT_PASSWORD=$${MINIO_ROOT_PASSWORD}
39     volumes:
40       - minio_data:/data
41     networks:
42       - appnet
43
44   volumes:
45     postgres_data:
46     minio_data:
47
48   networks:
49     appnet:
50
```

Açıklama: Sistem servislerinin orkestrasyonunu sağlayan compose dosyasının içeriği. web, db (PostgreSQL) ve minio servisleri; port yönlendirmeleri, named volume kullanımları, ortam değişkenleri ve ortak köprü ağı (appnet) yapılandırması açıkça görülmektedir.

Görüntü 4: docker compose up --build Komut Çıktısı

```
#13 resolving provenance for metadata file
#13 DONE 0.0s
[+] up 4/4
✓ Image imagerepository-web Built
✓ Container imagerepository-minio-1 Running
✓ Container imagerepository-db-1 Running
✓ Container imagerepository-web-1 Recreated
Attaching to db-1, minio-1, web-1
web-1 | * Serving Flask app 'app'
web-1 | * Debug mode: on
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.19.0.4:5000
web-1 | Press CTRL+C to quit
web-1 | * Restarting with stat
web-1 | * Debugger is active!
web-1 | * Debugger PIN: 144-028-488
[]
View in Docker Desktop View Config View Logs Enable Watch Detach
```

Açıklama: docker compose up --build komutunun başarıyla çalıştığını gösteren terminal ekran görüntüsüdür. Tüm servis imajlarının sorunsuz bir şekilde derlendiği, container birimlerinin oluşturulduğu ve servislerin Docker ağı üzerinde senkronize ayağa kalktığı doğrulanmıştır.

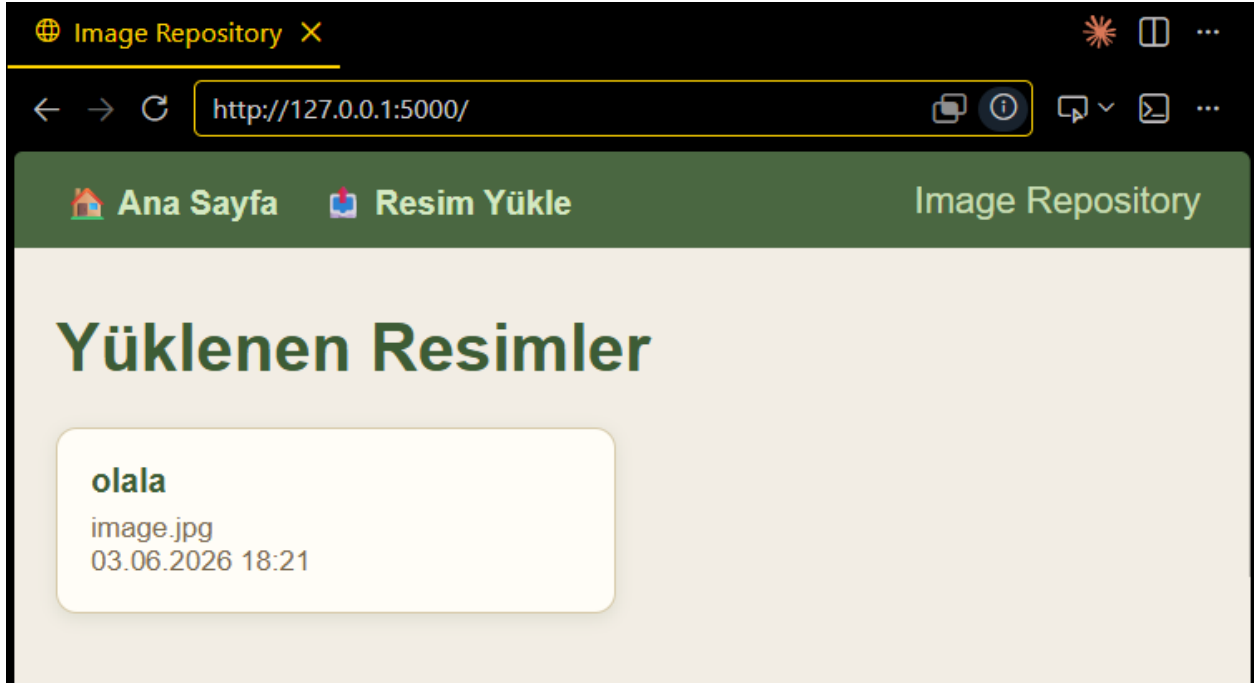
Görüntü 5: Aktif Container Durumları (ps)

```
C:\Users\NİSANUR\Documents\GitHub\imagerepository>docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
imagerepository-db-1	postgres:16	"docker-entrypoint.s..."	db	51 minutes ago	Up 29 minutes	5432/tcp
imagerepository-minio-1	minio/minio:latest	"/usr/bin/docker-ent..."	minio	51 minutes ago	Up 29 minutes	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
imagerepository-web-1	imagerepository-web	"python app.py"	web	29 minutes ago	Up 29 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

Açıklama: docker compose ps komutunun çıktısıdır. Web uygulaması (web), PostgreSQL veritabanı (db) ve MinIO nesne depolama (minio) servislerinin aktif olarak sorunsuz çalıştığı (Up durumu) ve dış dünyaya açılan port haritalamaları net bir şekilde kanıtlanmaktadır.

Görüntü 6: Web Uygulaması Ana Sayfası



Açıklama: Web uygulamasının tarayıcı üzerindeki ilk açılış (Ana Sayfa) görünümüdür. Tarayıcı adres çubuğunda kullanılan localhost portu net bir şekilde görünmekte ve henüz sisteme kayıtlı bir resim bulunmadığı bilgisi dinamik olarak listelenmektedir.

Görüntü 7: Resim Yükleme Formu

The screenshot shows a web browser window with the address bar displaying 'http://127.0.0.1:5000/upload'. The page has a dark green header with the text 'Image Repository' and a navigation bar with 'Ana Sayfa' (Home) and 'Resim Yükle' (Upload Image). The main content area is titled 'Resim Yükle' and contains a form with three sections: 'Başlık *' (Title) with a text input field containing 'Resim başlığı'; 'Açıklama' (Description) with a text area containing 'İsteğe bağlı açıklama'; and 'Resim Dosyası *' (Image File) with a 'Choose File' button and the text 'No file chosen'. A green 'Yükle' (Upload) button is at the bottom of the form.

Açıklama: Kullanıcının web istemci arayüzü üzerinden sisteme yeni bir görsel eklemesini sağlayan yükleme formudur. Başlık alanı, açıklama metni alanı ve zorunlu tutulan dosya seçme alanları eksiksiz yer almaktadır.

Görüntü 8: Başarılı Resim Yükleme Sonrası Arayüz



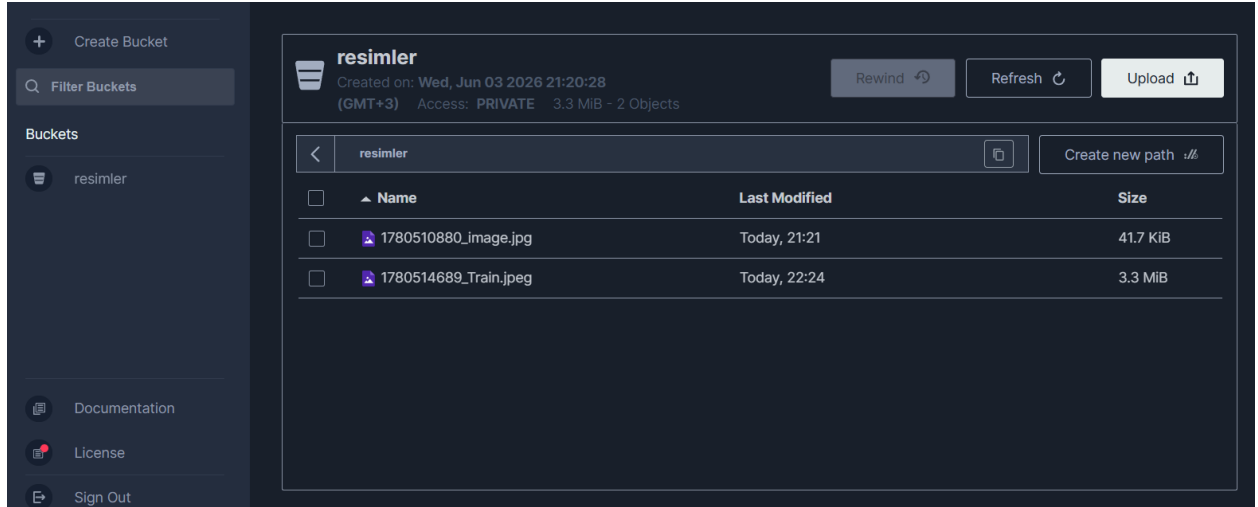
Açıklama: Başarılı resim yükleme işleminin ardından otomatik olarak güncellenen web arayüzüdür. Eklenen görsellerin başlık, orijinal dosya adı ve yüklenme tarihi bilgileriyle birlikte listelendiği gösterilmektedir.

Görüntü 9: Resim Detay ve Metadata Sayfası



Açıklama: Yüklenen bir resme ait detay ekranıdır. Görsel uygun boyutta ekranda render edilmekte, hemen altında PostgreSQL veritabanından çekilen başlık, açıklama, dosya adı, boyutu, türü, benzersiz MinIO Object Key değeri ve tam indirme butonu yer almaktadır.

Görüntü 10: MinIO Console Arayüz Doğrulaması



Açıklama: localhost:9001 adresi üzerinden erişilen MinIO Console yönetim paneli ekranıdır. Yapılandırma kapsamında kod tarafından otomatik üretilen 'resimler' bucket yapısı ve bu bucket içine object storage mimarisiyle kaydedilen görseller açıkça görülmektedir.

Görüntü 11: PostgreSQL Metadata Kayıtları

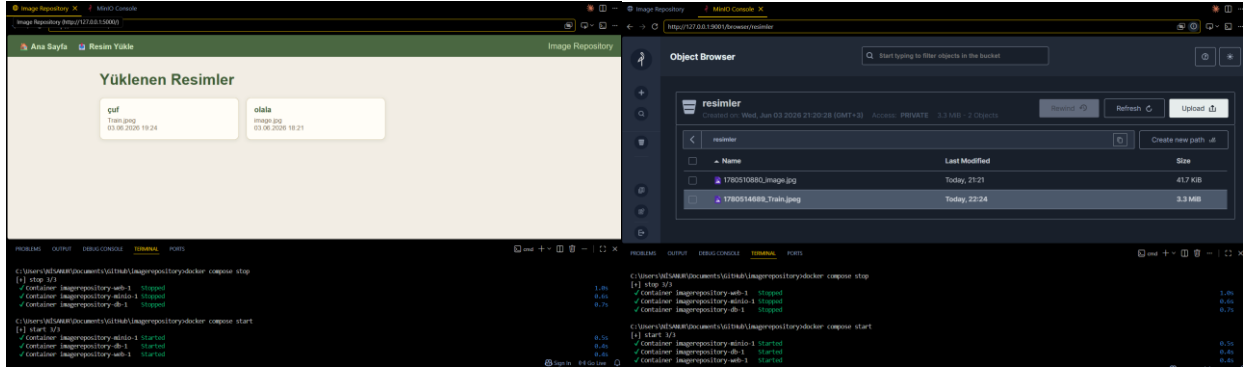
```
C:\Users\NİSANUR\Documents\GitHub\imagerepository>docker compose exec db psql -U ornek_kullanici -d image_repo_db -c "SELECT * FROM images;"
```

id	title	description	filename	filetype	filesize	object_key	uploaded_at
1	olala	alalo	image.jpg	jpg	42663	1780510880_image.jpg	2026-06-03 18:21:20.911476+00
2	çuf	çufçuf	Train.jpeg	jpeg	3465486	1780514689_Train.jpeg	2026-06-03 19:24:49.694958+00

(2 rows)

Açıklama: Terminal üzerinden psql aracı yardımıyla çalıştırılan SELECT sorgusu sonucudur. Resimlerin fiziki içeriklerinin ilişkisel tabloda yer almadığı, buna karşın her görsel nesneye ait tüm metadata kayıtlarının tabloya başarıyla işlendiği kanıtlanmıştır.

Görüntü 12: Volume Veri Kalıcılığı Kanıtı (Persistence)



Açıklama: İsimlendirilmiş birimlerin (Named Volume) kalıcılık kanıtıdır. Container yapıları terminalden stop edilip yeniden başlatıldıktan sonra dahi web arayüzündeki kayıtların, MinIO'daki fiziki resimlerin ve veritabanındaki metadata'nın tamamen korunduğu ispatlanmıştır.

Görüntü 13: Sistemdeki Named Volume Yapıları

```
C:\Users\NİSANUR\Documents\GitHub\imagerepository>docker volume ls
DRIVER      VOLUME NAME
local       imagerepository_minio_data
local       imagerepository_postgres_data
local       social-media-platform_mysql_data
```

Açıklama: Terminal ortamında docker volume ls komutunun çalıştırılmasıyla elde edilen çıktıdır. PostgreSQL verileri için oluşturulan postgres_data birimi ile MinIO verileri için oluşturulan minio_data isimlendirilmiş local birimlerinin varlığı tescillenmiştir.

Görüntü 14: Servislerin Güvenli Kapatılması (Down)

```
C:\Users\NİSANUR\Documents\GitHub\imagerepository>docker compose down
[+] down 4/4
✓ Container imagerepository-web-1    Removed
✓ Container imagerepository-db-1     Removed
✓ Container imagerepository-minio-1  Removed
✓ Network imagerepository_appnet     Removed
```

Açıklama: docker compose down komutu ile tüm projenin temiz bir şekilde sonlandırıldığını gösteren çıktıdır. Çalışan aktif container birimleri, bağımlı servis süreçleri ve oluşturulan sanal köprü ağı (appnet) sistem kaynaklarından güvenli kaldırılmıştır.